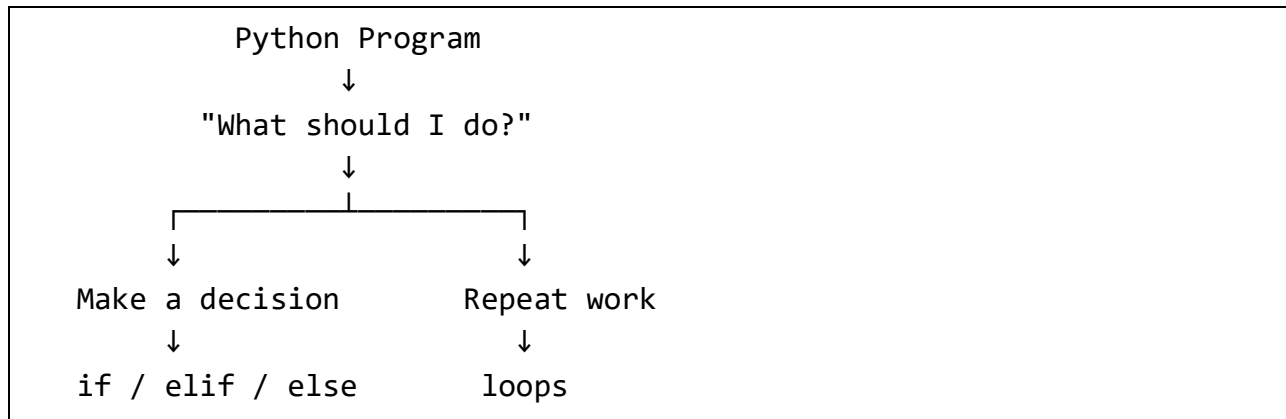


AIM2- Preparations

DAY 3 — Control Flow & Loops

The core idea is:



By the end of Day 3, I want you to understand:

- Comparison operators
- `if`
- `elif`
- `else`
- Logical operators
- `for`
- `range()`
- `while`
- `break`
- `continue`
- Nested conditions/loops
- How to combine these with your Student Gradebook

1 Comparison Operators

You already used this yesterday:

```
average >= 60
```

Python compares two values and gives you:

```
True  
False
```

Here are the important operators:

Operator	Meaning	Example
==	equal	5 == 5 → True
!=	not equal	5 != 3 → True
>	greater than	5 > 3 → True
<	less than	3 < 5 → True
>=	greater than/equal	5 >= 5 → True
<=	less than/equal	3 <= 5 → True

Very important

Don't confuse:

=

with:

==

= means assignment:

```
age = 28
```

You're saying:

Store 28 inside age.

== means comparison:

```
age == 28
```

You're asking:

Is age equal to 28?

2 if

You already used this:

```
average = 82.67

if average >= 60:
    print("PASSED")
```

Python asks:

Is average >= 60 True?

Yes → execute the indented code.

3 Indentation — VERY IMPORTANT

Python uses indentation to determine which code belongs inside a block.

Correct:

```
if average >= 60:
    print("PASSED")
```

The print() is indented.

This is different from Java.

Java

```
if (average >= 60) {  
    System.out.println("PASSED");  
}
```

Python

```
if average >= 60:  
    print("PASSED")
```

Python doesn't use {} for this.

Instead, it uses **indentation**.

Usually:

4 spaces

4 else

If the condition is false:

```
average = 45  
  
if average >= 60:  
    print("PASSED")  
else:  
    print("FAILED")
```

Output:

FAILED

Think:

IF condition is True



do A

ELSE



do B

5 elif

What if we have more than two possibilities?

For example, grades:

90+ → A

80+ → B

70+ → C

60+ → D

<60 → F

We use elif.

```
score = 85

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

print(grade)
```

Output:

B

How Python evaluates this

```
score = 85
```

```
85 >= 90 ❌
```

↓

```
85 >= 80 ✅
```

↓

```
grade = B
```

↓

```
STOP
```

It doesn't continue checking the remaining `elif`s once it finds a true condition.

Important example

Look carefully:

```
score = 95
```

```
if score >= 60:
```

```
    print("D")
```

```
elif score >= 80:
```

```
    print("B")
```

```
elif score >= 90:
```

```
    print("A")
```

What happens?

D

Why?

Because Python checks:

```
95 >= 60
```

```
True.
```

So it executes "D" and stops.

Therefore, when using ranges like grades, always put the **highest condition first**:

```
if score >= 90:  
    ...  
elif score >= 80:  
    ...  
elif score >= 70:  
    ...
```

This is a very important programming concept.

Logical Operators

Now let's make our conditions smarter.

Python has:

```
and  
or  
not
```

and

Both conditions must be true.

```
age = 25  
score = 85
```

```
if age >= 18 and score >= 60:  
    print("Eligible")
```

Think:

```
age >= 18      → True  
score >= 60    → True  
               ↓  
              AND  
               ↓  
             True
```

If either one is false:

True AND False → False

7 or

At least one condition must be true.

```
day = "Saturday"
```

```
if day == "Saturday" or day == "Sunday":  
    print("Weekend")
```

Here:

```
Saturday == Saturday → True  
Sunday == Saturday   → False
```

True OR False → True

not

not reverses the boolean.

```
is_student = True  
  
print(not is_student)
```

Output:

False

Another example:

```
is_logged_in = False  
  
if not is_logged_in:  
    print("Please login")
```

Quick Practice #1

Before learning loops, create a small program:

Enter your score: 85

Then output:

Grade: B

Use:

90 → A

80 → B

70 → C

60 → D

below 60 → F

Don't copy my earlier code. Write it yourself.

Now the BIG topic — for loops

Yesterday you used:

```
for student in students:
```

Today we're going to understand **why it works**.

Suppose:

```
students = ["Krishna", "Rahul", "Priya"]
```

Without a loop:

```
print(students[0])  
print(students[1])  
print(students[2])
```

With a loop:

```
for student in students:  
    print(student)
```

Output:

```
Krishna  
Rahul  
Priya
```

Python essentially does:

```
student = Krishna → print  
student = Rahul   → print  
student = Priya   → print
```

That's the fundamental idea of a loop:

Repeat something for each item.

10 for + list

```
scores = [95, 88, 65, 91]

for score in scores:
    print(score)
```

Output:

```
95
88
65
91
```

1 1 for + condition

Now we combine yesterday's knowledge with today's.

```
scores = [95, 88, 45, 72, 55]

for score in scores:
    if score >= 60:
        print(score, "PASSED")
    else:
        print(score, "FAILED")
```

Output:

```
95 PASSED
88 PASSED
45 FAILED
```

72 PASSED

55 FAILED

🔥 This is the kind of programming thinking I want you to develop.

You are now doing:

List

↓

Loop

↓

Condition

↓

Decision

1 2 range()

Sometimes we don't have a list.

We simply want to repeat something a certain number of times.

That's where range() comes in.

```
for i in range(5):  
    print(i)
```

Output:

0

1

2

3

4

Notice:

5 isn't included.

Exactly like slicing:

start included

end excluded

1 3 range(start, stop)

```
for i in range(1, 6):  
    print(i)
```

Output:

1
2
3
4
5

So:

range(1, 6)

means:

1 → included
2
3
4
5 → included
6 → excluded

1 4 range(start, stop, step)

You can also specify how much to increase each time.

```
for i in range(0, 11, 2):  
    print(i)
```

Output:

```
0  
2  
4  
6  
8  
10
```

Think:

```
start = 0  
stop = 11  
step = 2
```

1 5 Countdown

You can even go backwards.

```
for i in range(5, 0, -1):  
    print(i)
```

Output:

```
5  
4  
3
```

2
1

1 6 Nested loops

A loop inside another loop.

Example:

```
for i in range(3):  
    for j in range(2):  
        print(i, j)
```

Output:

```
0 0  
0 1  
1 0  
1 1  
2 0  
2 1
```

Don't worry if nested loops feel confusing.

Just understand:

```
outer loop  
  ↓  
  inner loop  
  ↓  
  inner loop finishes  
  ↓  
outer loop moves to next iteration
```

We'll practice these later.

1 7 while loop

A **for loop** is usually:

Repeat for each item / known number of times.

A **while loop** is:

Keep repeating **while a condition is true.**

Example:

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Output:

1
2
3
4
5

Notice:

```
count += 1
```

This means:

```
count = count + 1
```

Without that, we'd have:

```
count = 1
```

```
while count <= 5:
    print(count)
```

🤖 Infinite loop.

Because count never changes.

1 8 break

break means:

Stop the loop completely.

Example:

```
for number in range(1, 10):  
    if number == 5:  
        break  
  
    print(number)
```

Output:

1
2
3
4

When Python reaches:

`number == 5`

it says:

Stop the loop.

1 9 continue

continue means:

Skip this iteration and continue with the next one.

Example:

```
for number in range(1, 6):  
    if number == 3:  
        continue  
  
    print(number)
```

Output:

1
2
4
5

3 was skipped.

So remember:

```
break  
↓  
STOP THE LOOP  
  
continue  
↓  
SKIP THIS ITERATION
```

★ 2 0 Now let's combine everything

This is where your Day 3 knowledge starts becoming useful.

Suppose:

```
students = [  
    {"name": "Krishna", "score": 92},  
    {"name": "Rahul", "score": 75},  
    {"name": "Priya", "score": 58}  
]
```

We can do:

```
for student in students:  
  
    score = student["score"]  
  
    if score >= 90:  
        grade = "A"  
    elif score >= 80:  
        grade = "B"  
    elif score >= 70:  
        grade = "C"  
    elif score >= 60:  
        grade = "D"  
    else:  
        grade = "F"  
  
    print(student["name"], "→", grade)
```

Output:

```
Krishna → A  
Rahul → C  
Priya → F
```

Look at what you just combined:

List
↓
Dictionary
↓
for loop
↓
Dictionary access
↓
Variable
↓
if / elif / else
↓
Comparison operators

🔥 This is exactly why we are learning these topics in this order.

Exercise 1 — Grade Calculator

Ask the user:

Enter your score:

Then calculate:

90-100 → A

80-89 → B

70-79 → C

60-69 → D

0-59 → F

Example:

Enter your score: 87

Score: 87

Grade: B

Exercise 2 — Even and Odd

Create:

```
numbers = [10, 23, 44, 51, 62, 77, 80, 91]
```

Use a for loop and print:

10 → Even

23 → Odd

44 → Even

...

Hint:

You need the **modulus operator** you learned on Day 1:

`number % 2`

★ Exercise 3 — Student Gradebook Upgrade

This is the important one.

Take your Day 2 students:

```
students = [  
    {  
        "name": "Krishna",  
        "math": 95,  
        "python": 88,  
        "ai": 65  
    },  
    {  
        "name": "Rahul",
```

```
        "math": 82,  
        "python": 91,  
        "ai": 78  
    },  
    {  
        "name": "Priya",  
        "math": 90,  
        "python": 94,  
        "ai": 89  
    }  
]
```

Calculate the average for each student.

Then assign:

90+ → A

80+ → B

70+ → C

60+ → D

<60 → F

Your output should look approximately like:

```
=====
      STUDENT GRADEBOOK
=====
```

```
Name: Krishna  
Total: 248  
Average: 82.67  
Grade: B  
Result: PASSED
```

```
-----
```

```
Name: Rahul  
Total: 251  
Average: 83.67  
Grade: B
```

Result: PASSED

Name: Priya

Total: 273

Average: 91.00

Grade: A

Result: PASSED

★ Bonus

Add this:

Highest Scorer: Priya

Highest Average: 91.00

Your Day 3 Python knowledge now looks like this

